

Lean DSL Project Pages — Full Scaffold

Lean DSL Project Pages - Full Scaffold

This document explains how the Lean DSL, IR encoder, IR decoder, and transpiler layers are organized into individual project pages. Each page corresponds to one Lean module in the project structure.

1. Polarity

Defines the balanced ternary polarity used throughout the IR.

```
inductive Polarity
| pos
| zero
| neg
```

2. EntityId

Defines identifiers for all mathematical objects.

```
inductive EntityId
| var : String → EntityId
| num : Int → EntityId
| term : Nat → EntityId
| eqn : EntityId
| fn : Nat → EntityId
| mat : Nat → EntityId
| vec : Nat → EntityId
| set : Nat → EntityId
| rv : Nat → EntityId
| op : Nat → EntityId
| obj : Nat → EntityId
| mor : Nat → EntityId
| custom : String → EntityId
```

3. Entity

Defines the core entity structure.

```
structure Entity :=  
  (class : String := "E")  
  (polarity : Polarity)  
  (id : EntityId)  
  (chain : String := "*")
```

4. Attribute

Defines metadata attached to entities.

```
structure Attribute :=  
  (class : String := "A")  
  (polarity : Polarity)  
  (target : EntityId)  
  (key : String)  
  (value : String)  
  (chain : String := "*")
```

5. RelationOp

Defines all relation operators.

```
inductive RelationOp  
  | mul | add | sub | div | eq  
  | diff | integrate | limit | eval_at  
  | matmul | dot | cross | transpose | inv | det  
  | union | intersect | set_sub | subset  
  | and | or | not | implies | iff  
  | prob | expect | var | dist  
  | compose | id | assoc  
  | custom : String → RelationOp
```

6. OperationOp

Defines all operation operators.

```
inductive OperationOp  
  | add | sub | div | iso  
  | apply_diff_rule | apply_int_rule | change_var | limit_simplify  
  | row_reduce | diag | eig_decomp  
  | simplify_logic | simplify_set | normalize_formula  
  | custom : String → OperationOp
```

7. Relation

Defines relation tokens.

```
structure Relation :=  
  (class : String := "R")  
  (polarity : Polarity)  
  (op : RelationOp)  
  (e1 : Entity)  
  (e2 : Entity)  
  (e3 : Entity)  
  (chain : String := "*")
```

8. Operation

Defines operation tokens.

```
structure Operation :=  
  (class : String := "O")  
  (polarity : Polarity)  
  (op : OperationOp)  
  (e1 : Entity)  
  (e2 : Entity)  
  (chain : String := "*")
```

9. Graph

Defines the full graph structure.

```
structure Graph :=  
  (entities : List Entity)  
  (attributes : List Attribute)  
  (relations : List Relation)  
  (operations : List Operation)
```

10. Encoder

Defines the IR encoder.

```
def encodeGraph (g : Graph) : List String :=  
  let es := g.entities.map encodeEntity  
  let as := g.attributes.map encodeAttribute  
  let rs := g.relations.map encodeRelation  
  let os := g.operations.map encodeOperation  
  es ++ as ++ rs ++ os
```

11. Decoder

Defines the IR decoder.

```
def decodeGraph (tokens : List String) : Graph :=  
  tokens.foldl step { entities := [], attributes := [], relations := [], operations := [] }
```

12. Transpiler

Defines the Lean $\rightleftarrows$ Graph DSL $\rightleftarrows$ IR pipeline.

```
structure Transpiler (α : Type) :=  
  (toGraph : α → Graph)  
  (toTokens : α → List String)  
  (fromGraph : Graph → Option α)  
  (fromTokens : List String → Option α)
```
